Audio Device Internals

How Audio Data Flows from Application to Speaker – 7 Phases

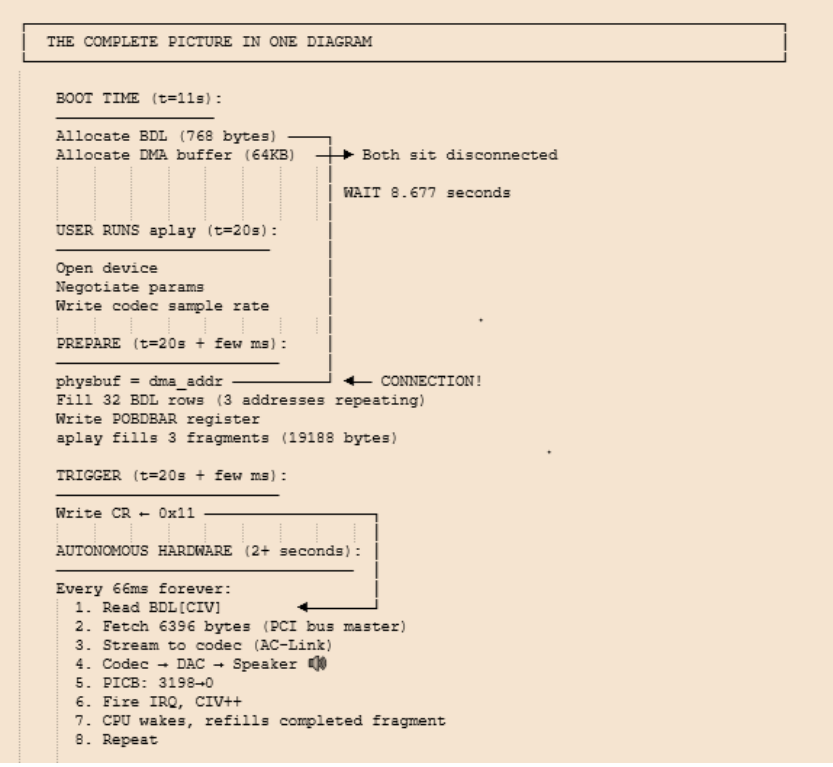
How Audio Plays on a Linux System (a quick overview)

When you press play, two things already exist in RAM from the moment the driver loaded at boot — a small 768-byte address table called the BDL (Buffer Descriptor List) and a 64 KB audio buffer (DMA buffer) where actual sound samples will live. Both are empty and have no connection to each other yet.

The moment aplay runs, the kernel negotiates the audio format (48000 Hz, stereo, 16-bit), writes the agreed sample rate to the AD1980 codec chip, then connects the two allocations by filling the BDL with the audio buffer's physical addresses. aplay then loads the first audio data from the WAV file directly into that buffer using a plain CPU memory copy.

From there, one byte written to a hardware register (CR = 0x11) starts the ICH DMA engine. The hardware takes over completely — it reads the BDL to find where audio is in RAM, fetches it over the PCI bus, streams it to the codec, and sends analog voltage to the speaker. The CPU is only involved once every 66 milliseconds to refill the finished audio fragment and keep the ring going. Everything else is hardware.

The seven phases below trace each of these steps in detail — with the exact kernel code that causes it and the live dmesg output that proves it happened.



PHASE 1: BDL Allocated at Probe

What Is Happening

When the driver loads (insmod), before any audio is ever played, it allocates a 768-byte table in RAM called the Buffer Descriptor List (BDL). Think of it as an address book with 32 empty rows. Each row has two fields: the physical address of an audio chunk in RAM, and how many samples that chunk holds. All 32 rows are zero at this point because the driver does not yet know where the audio data will live.

The 32-row limit is not a software choice. The ICH's LVI (Last Valid Index) register is exactly 5 bits wide (maximum value 31 = binary 11111). Hardware is hardwired to support entries 0 through 31 — 32 slots total. The driver cannot use more even if it wanted to.

One contiguous 768-byte block is allocated and mathematically divided into three 256-byte slices — one for PCM playback (PCMOUT at 0x37379100), one for PCM capture (PCMIN at 0x37379000), and one for the microphone (MIC at 0x37379200). Only the PCMOUT slice is relevant for playback.

Code: intel8x0.c

```
/* snd_intel8x0_init()  line 4236 */
chip->bdbars = snd_devm_alloc_pages(&pci->dev, intel8x0_dma_type(chip),
                chip->bdbars_count * sizeof(u32) * ICH_MAX_FRAGS * 2);
/* = 3 streams x 4 bytes x 32 entries x 2 DWords = 768 bytes */

/* Per-stream slice assignment  line 4247 */
ichdev->bdbar      = ((__le32 *)chip->bdbars->area) + (2 * 32 * i);
ichdev->bdbar_addr = chip->bdbars->addr + (2 * 32 * sizeof(u32) * i);
/* ichdev->bdbar      = virtual ptr   (CPU writes BDL entries here)      */
/* ichdev->bdbar_addr = physical addr (written to BDBAR hardware register) */
```

dmesg Proof:

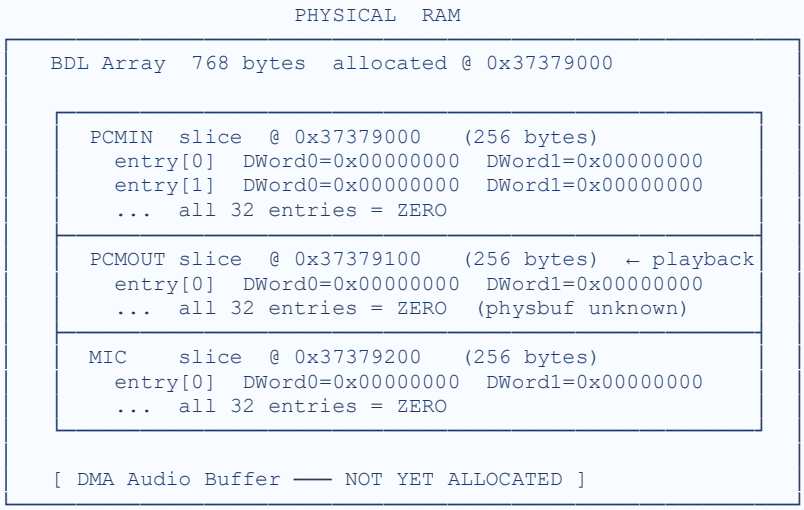
```
[  11.585s] TRACE-1: ONE contiguous block allocated
[  11.585s]   Function : snd_devm_alloc_pages()
[  11.585s]   Called at: PROBE time (not open, not prepare)
[  11.585s]   Formula  : streams x sizeof(u32) x 32 x 2
[  11.585s]   = 3 x 4 x 32 x 2 = 768 bytes total
[  11.585s]   Physical start: 0x37379000
[  11.585s]   Virtual  start: 000000005a396845

[  11.585s] TRACE-2: Block divided into 3 streams
[  11.585s]   stream[1] = ICHD_PCMOUT  phys: 0x37379100  (offset +256 from block
start)

[  11.585s] TRACE-3: Why always 32 entries?
[  11.585s]   ICH_REG_LVI_MASK = 0x1f = 31  (LVI register is 5 bits wide)
[  11.585s]   32 is NOT a software choice - Intel hardwired this in silicon

[  11.585s] TRACE-4: Pointer assigned - entries are ZERO
[  11.585s]   PCMOUT bdbar=0x37379100:
[  11.585s]     entry[0] DWord0=0x00000000  DWord1=0x00000000  <- ZERO (not filled
yet)
[  11.585s]     entry[1] DWord0=0x00000000  DWord1=0x00000000  <- ZERO (not filled
yet)
[  11.585s]   BDL CANNOT be filled yet - physbuf not known until prepare()
```

DIAGRAM — Phase 1: BDL Array Allocated in RAM (entries all zero)



PHASE 2: DMA Audio Buffer Allocated at Probe

What Is Happening

Slightly after the BDL is created, the driver allocates a 64KB block of physical RAM to hold actual audio samples. This is completely separate from the BDL — it has nothing to do with the address table. It is allocated using dma_alloc_coherent(), which returns two addresses for the exact same physical memory:

- cpu_addr (virtual: 0x605fae8c) — the CPU uses this when writing audio data into the buffer
- dma_handle (physical: 0x64210000) — the hardware uses this when fetching audio over the PCI bus

The buffer must be coherent (cache-bypassed). Normally when the CPU writes to memory the data lands in L1/L2 cache first, not directly in RAM. The ICH hardware reads RAM directly, bypassing CPU cache entirely. If normal cached memory were used, the hardware would read stale data from RAM while new audio sat in cache. Coherent memory marks those pages as uncacheable so every CPU write goes straight to physical RAM and hardware always reads the freshest audio data.

After this step, both allocations exist in RAM — the 768-byte BDL table and the 64KB audio buffer — but they have no connection to each other. In the dmesg collected here, BDL was at t=11.585s and DMA buffer at t=11.853s. The user ran aplay at t=20.530s. That is an 8.677-second gap where both sat idle doing nothing.

Code: intel8x0.c

```
/* snd_intel8x0_pcm_new()  line 2600 */
snd_pcm_set_managed_buffer_all(pcm, intel8x0_dma_type(chip),
```

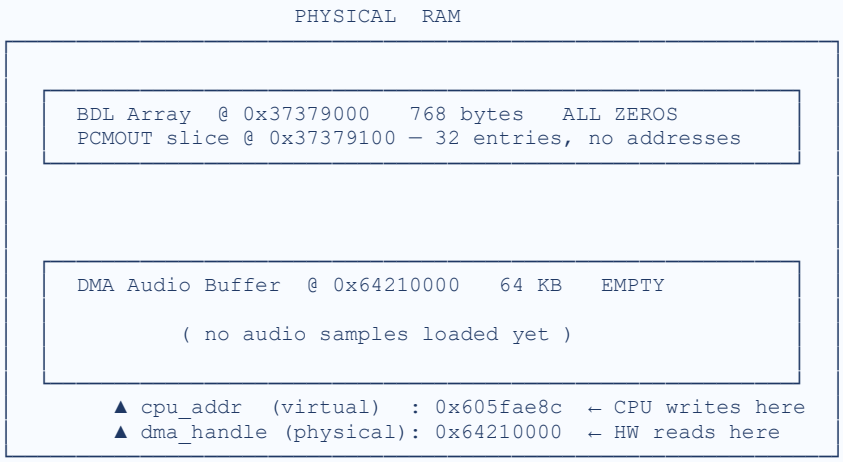
```
                                &chip->pci->dev,
                                rec->prealloc_size,       /* 64 KB */
                                rec->prealloc_max_size);  /* 128 KB */

/* 5-level call chain down to the actual allocation: */
/* L1: snd_pcm_set_managed_buffer_all()  <- intel8x0.c              */
/* L2: snd_pcm_set_managed_buffer()      <- sound/core/pcm_memory.c */
/* L3: preallocate_pages()               <- sound/core/pcm_memory.c */
/* L4: snd_dma_alloc_pages()             <- sound/core/memalloc.c   */
/* L5: dma_alloc_coherent()              <- ACTUAL allocation        */
/*       returns cpu_addr   = virtual  (CPU writes here)            */
/*               dma_handle = physical (hardware reads here via PCI) */
```

Dmesg:

```
[  11.853s] TRACE-1: DMA buffer allocated (PROBE TIME)
[  11.853s]   5-level chain: set_managed_buffer_all
[  11.853s]   -> set_managed_buffer -> preallocate_pages
[  11.853s]   -> snd_dma_alloc_pages -> dma_alloc_coherent
[  11.853s]   cpu_addr  (virtual) : 00000000605fae8c  <- CPU writes here
[  11.853s]   dma_handle(physical): 0x64210000         <- hardware reads here
[  11.853s]   size=65536 bytes - coherent (cache bypassed)
[  11.853s]   WHY coherent: CPU writes skip cache -> RAM directly
[  11.853s]   Hardware reads RAM not cache -> always latest audio
[  11.853s]   Buffer EMPTY - waiting for aplay
```

DIAGRAM — Phase 2: Both Allocations Exist But Are Disconnected



PHASE 3: open() and Format Negotiation

What Is Happening

The moment you press play, aplay opens the audio device file /dev/snd/pcmC0D0p. The driver responds by publishing a capability menu stating what it can support: S16_LE format only, 48000 Hz only, stereo only, period sizes from 32 bytes up to 128 KB. No hardware register is written at this stage.

The ALSA kernel layer (sound/core/pcm_native.c) then runs a constraint engine that negotiates between what aplay wants and what the driver supports. It settles on: period_size=1599 frames, periods=3, fragsize=6396 bytes per period, buffer_size=4797 frames total. These values are stored in the runtime struct.

In the dmesg collected here, this negotiation happened 7 times in 34ms (t=20.497s to t=20.530s) as the media player probed different channel counts (2, 6, 4, 6, 6, 6, 2) to discover what the hardware supports. Only the final attempt at t=20.530s with channels=2 proceeded all the way to trigger. The ICH DMA engine was completely idle throughout all 7 probes.

Code: intel8x0.c

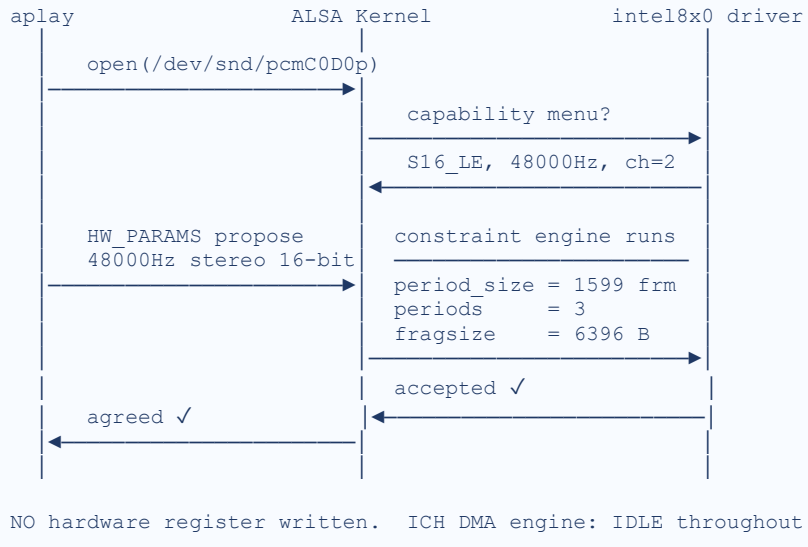
```
/* snd_intel8x0_pcm_open()  line 2211 */
runtime->hw = snd_intel8x0_stream;   /* publish capability menu to ALSA */
/* snd_intel8x0_stream contains:                                        */
/*   formats         = S16_LE only                                      */
/*   rate_min/max    = 48000 Hz only                                    */
/*   channels_min/max = 2                                               */
/*   period_bytes_min = 32,   period_bytes_max = 131072                 */
/*   periods_min = 1,         periods_max = 1024                        */

err = snd_pcm_hw_constraint_integer(runtime, SNDRV_PCM_HW_PARAM_PERIODS);
/* Constraint: periods must be a whole integer - no fractions allowed    */
```

dmesg Proof:

```
/* Final accepted session (channels=2) at t=20.530s */
[  20.530s] TRACE-2: open() - capability menu published
[  20.530s]   format=S16_LE  rate_min=48000  rate_max=48000
[  20.530s]   channels_min=2  channels_max=2
[  20.530s]   period_bytes_min=32  period_bytes_max=131072
[  20.530s]   periods_min=1  periods_max=1024
[  20.530s]   NO hardware register written at open()
[  20.530s]   ICH DMA engine: IDLE,  codec: not configured
```

DIAGRAM — Phase 3: open() Format Negotiation (no hardware touched)



PHASE 4: Format Agreed, Codec Written

What Is Happening

Once format negotiation succeeds, the driver's hw_params callback is called. Its job is to write the agreed sample rate to the AD1980 codec chip over the AC-Link serial bus. This is the very first real hardware register written in the entire session.

AC-Link is a serial protocol running at 12.288 MHz that connects the ICH controller to the AD1980 codec. The driver writes 48000 to the AC97_PCM_FRONT_DAC_RATE register. The AD1980 now knows: expect digital audio arriving at 48000 samples per second and convert each sample to analog voltage.

The ICH DMA engine is still completely idle at this point. No BDL entries have been filled. No audio data is in the DMA buffer. Only the codec side knows what format is coming.

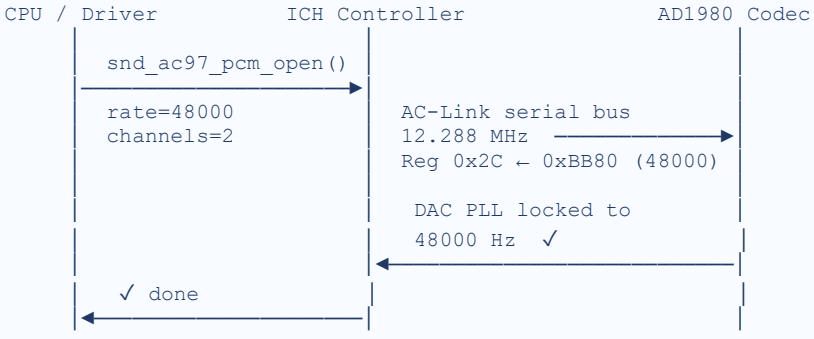
Code: intel8x0.c

```
/* snd_intel8x0_hw_params()  line 1777 */
err = snd_ac97_pcm_open(ichdev->pcm,
                        params_rate(hw_params),      /* 48000 */
                        params_channels(hw_params),  /* 2     */
                        ichdev->pcm->r[dbl].slots);
/* Writes AC97_PCM_FRONT_DAC_RATE = 48000 to AD1980 codec           */
/* over AC-Link serial bus (12.288 MHz, 256-bit frames at 48 kHz)   */
/* FIRST hardware register written in the entire session            */
```

Dmesg:

```
[  20.530s] TRACE-3: HW_PARAMS - format agreed
[  20.530s]   rate=48000  channels=2  format=S16_LE
[  20.530s]   AC97_PCM_FRONT_DAC_RATE <- 48000
[  20.530s]   AD1980 codec now knows rate
[  20.530s]   ICH DMA engine: still IDLE
[  20.530s]   BDL: still zero,  DMA buffer: still empty
```

DIAGRAM — Phase 4: First Hardware Register Written — AC-Link to AD1980 Codec




```
AD1980 now expects: PCM audio at 48000 samples/sec.
ICH DMA engine: still IDLE.  BDBAR, LVI, CIV, CR = 0x00.

Note: AC-Link is CONTROL path (slots 1+2) — different from
PCM DATA path (slots 3+4) used during actual audio playback.
```

PHASE 5: Prepare: BDL Filled, Audio Data Loaded

What Is Happening

This is the step where everything comes together. Three things happen in sequence:

1. The two allocations are connected.

One line of code links the BDL table to the DMA audio buffer: ichdev->physbuf = runtime->dma_addr. This is not a new allocation. It simply copies the physical address of the DMA buffer (0x64210000, known since probe) into the ichdev struct so the BDL fill loop can use it.

2. All 32 BDL rows are filled.

The fill loop writes all 32 rows. Each row gets a physical fragment address in DWord0, and in DWord1 it gets IOC=1 (bit 31, meaning fire an interrupt when this entry finishes) and the sample count 3198 (bits 14:0 = 6396 bytes / 2 bytes per sample). The hardware register POBDBAR is written with the BDL's physical address so the ICH knows where to find its address table. LVI is set to 31, telling the hardware all 32 entries are valid.

3. aplay loads audio data via copy_from_user().

aplay calls write() which internally calls copy_from_user(). This is a plain CPU memory copy — audio data goes from the WAV file in userspace directly into the DMA buffer fragments in kernel RAM. No DMA, no

codec involved. Just the CPU copying bytes. After this, the DMA buffer contains real audio samples, the BDL knows exactly where they are, and everything is loaded and ready. The hardware has still not started.

Code: intel8x0.c

```
/* snd_intel8x0_pcm_prepare()  line 1964 */
ichdev->physbuf  = runtime->dma_addr;            /* 0x64210000 */
ichdev->size     = snd_pcm_lib_buffer_bytes(substream); /* 19188 bytes */
ichdev->fragsize = snd_pcm_lib_period_bytes(substream); /* 6396 bytes  */
/* physbuf is NOT a new allocation.                                    */
/* runtime->dma_addr was set at probe time by dma_alloc_coherent().  */
/* This line just copies that address into ichdev for setup_periods. */

/* snd_intel8x0_setup_periods()  line 693 */
iputdword(chip, port + ICH_REG_OFF_BDBAR, ichdev->bdbar_addr);
/* POBDBAR register <- 0x37379100  (tells ICH where BDL lives)      */

for (idx = 0; idx < (ICH_REG_LVI_MASK + 1) * 2; idx += 2) {
    bdbar[idx+0] = cpu_to_le32(ichdev->physbuf +
                    (((idx>>1) * ichdev->fragsize) % ichdev->size));
    /* DWord0: physical address of the audio fragment              */
    bdbar[idx+1] = cpu_to_le32(0x80000000 |   /* bit31 = IOC = 1   */
                    ichdev->fragsize >> ichdev->pos_shift);
    /* DWord1: 0x80000C7E = IOC=1, samples=3198                    */
}
iputbyte(chip, port + ICH_REG_OFF_LVI, ichdev->lvi = ICH_REG_LVI_MASK);
/* LVI register <- 31  (all 32 entries valid, hardware may use all) */
```

Dmesg:

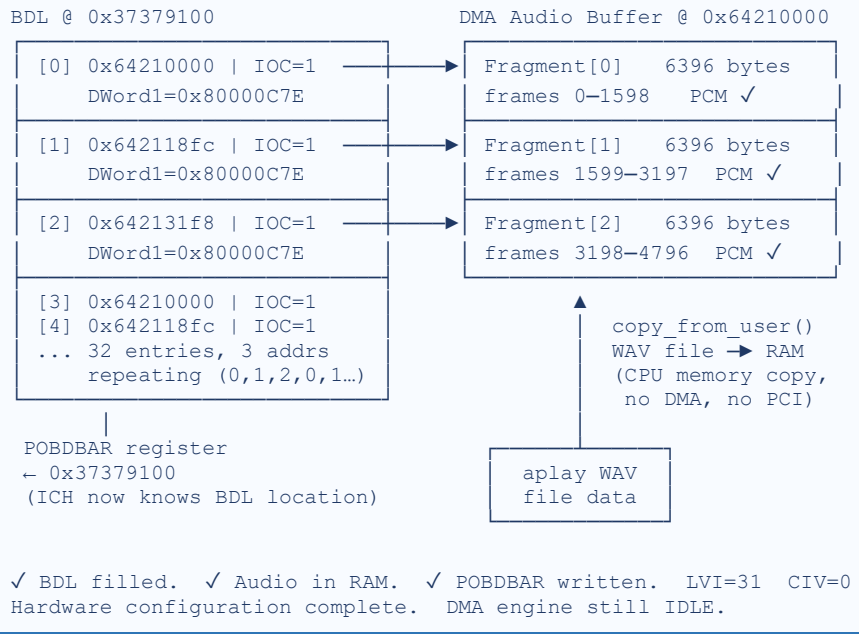
```
[  20.530s] TRACE-5: PREPARE reached - BDL can now be filled
[  20.530s]   runtime->dma_addr  = 0x64210000 (physical)
[  20.530s]   runtime->dma_area  = 00000000605fae8c (virtual)
[  20.530s]   runtime->dma_bytes = 19188 bytes
[  20.530s]   rate=48000  channels=2  sample_bits=16
[  20.530s]   period_size=1599 frames  periods=3  buffer_size=4797 frames

[  20.530s] TRACE-4: PREPARE - physbuf assigned (NOT new alloc)
[  20.530s]   ichdev->physbuf   = 0x64210000
[  20.530s]   runtime->dma_addr = 0x64210000  <- SAME address, no new
allocation
[  20.530s]   virt_to_phys(dma_area) = 0x64210000
[  20.530s]   Match: YES virtual=physical same RAM
[  20.530s]   total=19188 bytes  fragsize=6396 bytes  frags=3
[  20.530s]   Fragment[0]: 0x64210000  (offset +0)
[  20.530s]   Fragment[1]: 0x642118fc  (offset +6396)
[  20.530s]   Fragment[2]: 0x642131f8  (offset +12792)

[  20.530s] TRACE-6: DMA buffer divided + BDL filled
[  20.530s]   fragsize=6396 bytes  samples per frag=3198  (6396 >>
pos_shift(1))
[  20.530s]   [ 0] 0x64210000 | IOC=1 smpls=3198 -> Fragment[0] offset=+0
[  20.530s]   [ 1] 0x642118fc | IOC=1 smpls=3198 -> Fragment[1]
offset=+6396
```

```
[  20.530s]    [ 2] 0x642131f8 | IOC=1 smpls=3198 -> Fragment[2]
offset=+12792
[  20.530s]    [ 3] 0x64210000 | IOC=1 smpls=3198 -> Fragment[0] offset=+0
[  20.530s]    ... pattern repeats across all 32 entries ...
[  20.530s]    32 BDL slots filled.  Alignment: bdbar_addr & 7 = 0
(datasheet 12.2.1 OK)
```

DIAGRAM — Phase 5: BDL Filled + Audio Loaded — Connection Established



PHASE 6:Trigger: CPU Writes 0x11, Hardware Takes Over

What Is Happening

With the BDL filled and audio data in RAM, aplay sends the start command. The driver writes a single byte — 0x11 — to the POCR register (PCM Out Control Register). This one write starts the entire DMA engine.

0x11 in binary is 00010001. Bit 0 is STARTBM (start bus master) and bit 4 is IOCE (interrupt on completion enable). At this point CIV (Current Index Value) is at 0 (first entry) and LVI is already at 31 (all entries valid, set during prepare).

Inside the ICH silicon, the CR register is made of flip-flops — tiny circuits that hold a bit value and drive output wires. When bit 0 (STARTBM) latches to 1, the wire connected to the DMA state machine goes HIGH. The state machine is built from combinational logic that evaluates on every 33 MHz clock cycle. On the very next clock tick — approximately 30 nanoseconds — the state machine transitions from IDLE to

RUNNING. The CPU instruction that wrote 0x11 has already completed and is executing the next line of code. Hardware is now fully autonomous.

Reading back the CR register immediately after gives 0x11 (flip-flops latched correctly). Reading the Status Register shows DCH=0, confirming DMA is running (DCH=1 means halted)

Code: intel8x0.c

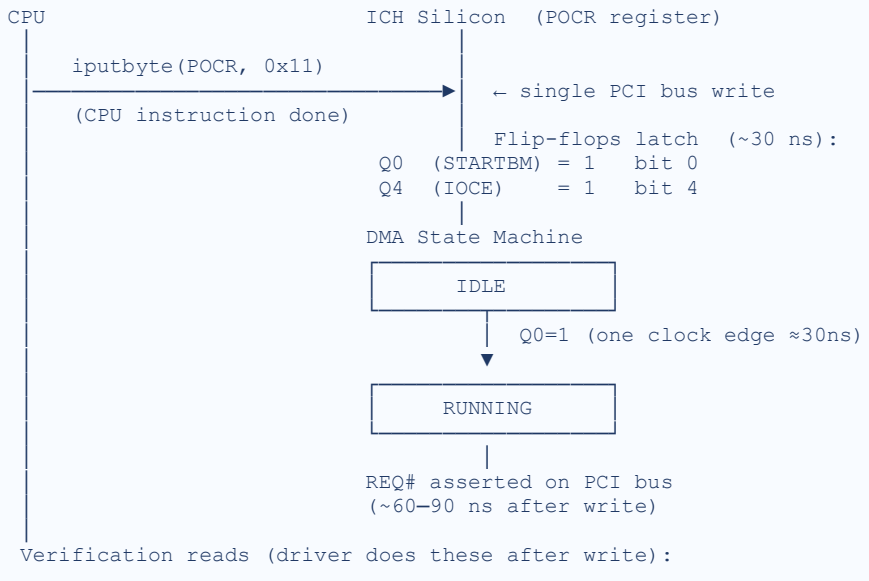
```
/* snd_intel8x0_pcm_trigger()  line 1582 */
case SNDRV_PCM_TRIGGER_START:
    val = ICH_IOCE | ICH_STARTBM;  /* 0x10 | 0x01 = 0x11 */
    /* ICH_STARTBM = 0x01 = bit0: start busmaster DMA engine   */
    /* ICH_IOCE    = 0x10 = bit4: enable IRQ on completion     */
    break;

iputbyte(chip, port + ICH_REG_OFF_CR, val);  /* line 1617 */
/* POCR register <- 0x11                                       */
/* In silicon:                                                  */
/*   PCI bus carries 0x11 to ICH CR register address           */
/*   Flip-flops latch: Q0(STARTBM)=1, Q4(IOCE)=1               */
/*   DMA state machine sees Q0=1: IDLE -> RUNNING in ~60-90ns   */
/*   CPU instruction done. Hardware autonomous from this point. */
```

Dmesg:

```
[  20.545s] TRACE-5: TRIGGER - CPU writing CR=0x11
[  20.545s]   bit0 STARTBM=1  <- start DMA engine
[  20.545s]   bit4 IOCE=1     <- enable IRQs for this stream
[  20.545s]   Audio already in DMA buffer at 0x64210000
[  20.545s]   PCI bus carries 0x11 to CR flip-flops
[  20.545s]   Q0 latches=1 -> DMA state machine IDLE->RUNNING
[  20.545s]   ~60-90ns after write: ICH asserts REQ# on PCI bus
[  20.545s]   CR readback=0x11   <- flip-flops latched confirmed
[  20.545s]   SR=0x00  DCH=0     <- DMA RUNNING (not halted) confirmed
[  20.545s]   CPU done - hardware autonomous from here
```

DIAGRAM — Phase 6: CR=0x11 Written — DMA Engine Transitions IDLE → RUNNING



```
    CR readback = 0x11  ✓  (flip-flops latched correctly)
    SR = 0x00, DCH = 0  ✓  (DMA running, not halted)
```

PHASE 7: Hardware Runs Alone for the Entire Song

What Is Happening

From the moment CR=0x11 is written, the ICH chip runs completely independently. The CPU is free to do anything else. Every 66 milliseconds the following sequence repeats automatically:

- ICH reads the BDBAR register to find where the BDL table sits in physical RAM.
- ICH uses CIV (Current Index Value, starts at 0) to index into the BDL. First PCI bus access: just 8 bytes to read that BDL entry. Bus is released immediately afterwards.
- From those 8 bytes: DWord0 gives the physical address of the audio fragment. DWord1 gives the IOC bit (bit 31) and the sample count, which is loaded into PICB.
- Second PCI bus access: ICH fetches 6396 bytes of audio data from the fragment address in RAM into its internal FIFO. Bus released. PCI bus is now free for the remaining 66ms — network card, USB, GPU can all use it normally.
- ICH streams samples to AD1980 over AC-Link (12.288 MHz serial wire). AD1980 converts each digital number to analog voltage. Speaker cone moves. Sound comes out.
- PICB counts down every sample: 3198 -> 3197 -> 3196 -> ... -> 1 -> 0. CPU is NOT involved.
- PICB hits 0: ICH checks IOC bit. IOC=1, so SR.BCIS=1 is set and INTA# is pulled LOW on the PCI bus. This is the interrupt signal to the CPU.
- Hardware does NOT wait. It immediately increments CIV (0->1), reads the next BDL entry, fetches the next audio fragment, and starts playing. Audio never stops.

Meanwhile the interrupt reaches the CPU. The handler snd_intel8x0_interrupt() is called, which calls snd_intel8x0_update(). By the time the CPU reads CIV it is already 1 — hardware moved on before the CPU even started handling the interrupt. This is the proof of true hardware autonomy.

The handler then: advances LVI by 1 so it stays ahead of CIV (if CIV ever catches LVI the hardware halts), clears SR.BCIS by writing 1 to it (write-1-to-clear), and calls snd_pcm_period_elapsed() which wakes aplay. aplay calls copy_from_user() to refill the finished fragment with the next chunk from the WAV file.

CIV increments 0->1->2->...->31, then wraps back to 0. LVI is always chased one lap ahead by the CPU handler. The physical RAM addresses never change for the entire song. Only the audio content inside them is refreshed every 66ms. When aplay runs out of data, the CPU writes 0x00 to the CR register — STARTBM goes LOW, the ICH state machine transitions RUNNING->IDLE, and silence.

Code: intel8x0.c

```
/* snd_intel8x0_update()  line 906  (called from interrupt handler) */
spin_lock_irqsave(&chip->reg_lock, flags);
status = igetbyte(chip, port + ichdev->roff_sr);   /* read SR register  */
civ    = igetbyte(chip, port + ICH_REG_OFF_CIV);   /* CIV already++ by HW */

/* Advance LVI to keep ring alive  line 1309 */
ichdev->lvi += step;
ichdev->lvi &= ICH_REG_LVI_MASK;          /* wrap at 31             */
iputbyte(chip, port + ICH_REG_OFF_LVI, ichdev->lvi);

/* Clear BCIS  line 1312 */
iputbyte(chip, port + ichdev->roff_sr, ICH_BCIS);  /* write-1-to-clear  */

/* Wake aplay to refill  line 1364 */
snd pcm period elapsed(ichdev->substream);
/* aplay: write() -> copy from user() -> next WAV chunk into RAM     */
```

Dmesg:

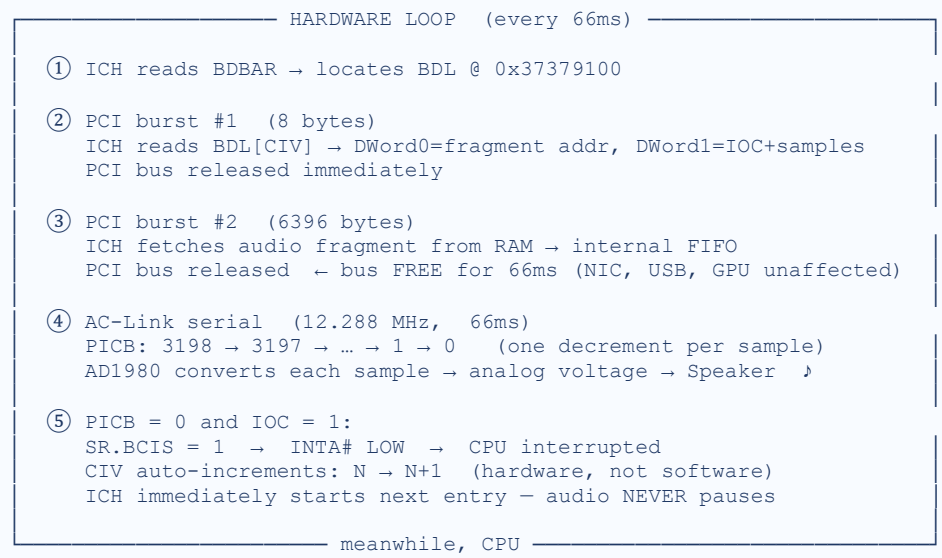
```
[  20.578s] TRACE-6: IRQ #1 autonomous proof
[  20.578s]   PCI burst#1: read BDL[CIV] 8 bytes -> released bus
[  20.578s]   PCI burst#2: read DMA 6396 bytes   -> released bus
[  20.578s]   AC-Link: PICB 3198->0  CPU NOT involved
[  20.578s]   CIV=1 (was 0, HW incremented before CPU ran)  LVI=31  PICB=3198
[  20.578s]   SR=0x08  BCIS=1  DCH=0
[  20.578s]   NEXT: period_elapsed wakes aplay -> copy_from_user refills

[  20.579s] TRACE-6: IRQ #2
[  20.579s]   CIV=1 (was 1)  SR=0x00  BCIS=0  <- SPURIOUS (IRQF_SHARED, IRQ 21)
[  20.579s]   Another device fired. Driver correctly ignored it.

[  20.612s] TRACE-6: IRQ #3 autonomous proof
[  20.612s]   PCI burst#1: read BDL[CIV] 8 bytes -> released bus
[  20.612s]   PCI burst#2: read DMA 6396 bytes   -> released bus
[  20.612s]   AC-Link: PICB 3198->0  CPU NOT involved
[  20.612s]   CIV=2 (was 1, HW incremented)  LVI=0  PICB=3198
[  20.612s]   SR=0x08  BCIS=1  DCH=0
[  20.612s]   NEXT: period_elapsed wakes aplay -> copy_from_user refills

/* Loop ran for 30 seconds (~450 IRQs).  Counter is global across sessions: */
[  284.156s] IRQ #500     [  292.485s] IRQ #1000    [  321.574s] IRQ #1500
[  330.530s] IRQ #2000    [  351.901s] IRQ #3000    [ 1540.406s] IRQ #4000
[ 1998.430s] IRQ #5000    [ 2015.364s] IRQ #6000
/* 6000+ IRQs total - hardware never stopped being autonomous             */
```

DIAGRAM — Phase 7: Hardware Autonomous Loop — ICH → AC-Link → AD1980 → Speaker



8. Complete Timeline — One Clean Session

```
t=11.585s  PROBE: snd_intel8x0_init()
           Code:  snd_devm_alloc_pages() -> 768 bytes at phys 0x37379000
           BDL:   3 slices, 32 entries all ZERO, waiting for physbuf

t=11.853s  PROBE: snd_intel8x0_pcm_new()
           Code:  snd_pcm_set_managed_buffer_all() -> dma_alloc_coherent()
           DMA:   64KB coherent, physical=0x64210000, virtual=0x605fae8c, EMPTY
           State: Both allocations exist but are DISCONNECTED from each other

           <--- 8.677 seconds idle: both allocations sitting unused --->

t=20.530s  USER: aplay -D hw:0,0 /tmp/test.wav
           Code:  snd_intel8x0_pcm_open()  -> publishes snd_intel8x0_stream
           DMSG:  TRACE-2: menu published, NO hardware register written

           Code:  snd_ac97_pcm_open() -> AC97_PCM_FRONT_DAC_RATE <- 48000
           DMSG:  TRACE-3: rate=48000 ch=2 agreed, FIRST hardware register written

           Code:  ichdev->physbuf = runtime->dma_addr  (line 1964)
           Code:  setup_periods() BDL fill loop        (line 693)
           DMSG:  TRACE-5: PREPARE reached, physbuf now known
           DMSG:  TRACE-4: physbuf=0x64210000 = dma_addr, virt_to_phys MATCH
           DMSG:  TRACE-6: 32 BDL entries filled, IOC=1, smpls=3198 on all
                           BDL <-> DMA buffer connection established

           [aplay: write() -> copy_from_user() fills DMA fragments with WAV data]

t=20.545s  TRIGGER: snd_intel8x0_pcm_trigger()
           Code:  val=ICH_IOCE|ICH_STARTBM=0x11; iputbyte(POCR, 0x11)  (line 1617)
           DMSG:  TRACE-5: CR=0x11 latched, IDLE->RUNNING, DCH=0 confirmed
                           CPU DONE. Hardware is autonomous.

t=20.578s  IRQ #1: snd_intel8x0_interrupt() -> snd_intel8x0_update()
           Code:  civ = igetbyte(CIV)             (line 921)  <- already 1
           Code:  lvi += step; iputbyte(LVI)       (line 1309)
           Code:  iputbyte(SR, ICH_BCIS)           (line 1312) write-1-to-clear
           Code:  snd_pcm_period_elapsed()         (line 1364) wake aplay
           DMSG:  TRACE-6: CIV=1 (was 0), BCIS=1, hardware autonomy proven

           [repeats every 66ms x ~450 IRQs for the 30-second test.wav]
```

9. Key Numbers and Registers

Item	Value	Notes
BDL physical (PCMOUT)	`0x37379100`	Written to POBDBAR register
DMA buffer physical	`0x64210000`	From dma_alloc_coherent() at probe
DMA buffer virtual	`0x605fae8c`	CPU writes here; same physical block
DMA buffer size	`64 KB (65536 B)`	Preallocated; 19188 bytes used
Fragsize	`6396 bytes`	1599 frames x 4 bytes/frame
Periods	`3`	buffer_size / fragsize
samples per fragment	`3198`	6396 >> pos_shift(1)
IRQ period	`~66 ms`	3198 samples / 48000 Hz
POCR / CR write value	`0x11`	STARTBM=1 (bit0), IOCE=1 (bit4)
LVI at trigger	`31`	ICH_REG_LVI_MASK; all 32 valid
CIV at trigger	`0`	Auto-incremented by hardware per entry
PICB initial	`3198`	Counts down to 0 per fragment
BDL total entries	`32`	LVI register is 5 bits wide (hardwired)
IOC bit	`bit31 of DWord1`	Set on every entry; fires IRQ at PICB=0
Total IRQs observed	`6000+`	Across multiple aplay sessions; static counter